



ADITYA UNIVERSITY

SCHOOL OF ENGINEERING

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Cornerstone project work entitled “**FACULTY LEAVE
MANAGEMENT SYSTEM**” is submitted by

M.SYAM PRASAD (25B21CS018)

P.PRABHAS (24B11CS338)

K.SATHEESHKUMAR (24B11CS473)

DARLINGTON (24B11CS538)

in partial fulfillment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering for the academic year 2025-2026.

Project Guide

Mrs R.Padma Sri,
Assistant Professor,
Department of CSE

Head of the Department

Dr.T.Sudha Rani,
Assistant Professor,
Department of CSE

DECLARATION

We hereby declare that the Cornerstone project entitled “**FACULTY LEAVE MANAGEMENT SYSTEM**” is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. degree in Computer Science and Engineering**. We further declare that this project work has not been submitted, either in full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy

Place: Surampalem

Date:

By

M.SYAM PRASAD	(25B21CS018)
P.PRABHAS	(24B11CS338)
K.SATHEESHKUMAR	(24B11CS473)
DORLINGTON	(24B11CS538)

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **Mrs. R.Padma Sri, Assistant Professor**, who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. K.Chandrasekhar, Associate Professor and Dr.Hari Jyothula, Associate Professor**, Department of CSE, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. T.Sudha Rani, Assistant Professor**, for her inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Engineering** for their encouragement and support during the course of our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama S r e e , Pro Vice-Chancellor (Academics), D r . M.B. Srinivas, V i c e -Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

Table of contents

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iii
ACRONYMS	iv
1. INTRODUCTION	1
1.1 Brief Information about the Project	1
1.2 Motivation and Contribution of Project	1
1.3 Objectives of the Project	2
1.4 Scope of the Project	2
2. SYSTEM ANALYSIS	3
2.1 Functional Requirements	3
2.2 Non-Functional Requirements	4
2.3 Requirements Specification	4
2.3.1 Minimum Hardware Requirements	4
2.3.2 Software Requirements	4
3. TECHNOLOGY DESCRIPTION	5
3.1 Programming Language	5
3.1.1 Introduction to JavaScript	5
3.2 Framework / Libraries	5
3.2.1 Introduction to React	5
3.2.2 Introduction to Node.js	6
3.3 Database Technology	6
3.3.1 Mongoose	7
3.3.2 MongoDB	7
3.4 Tools & IDEs	7
3.4.1 Visual Studio Code	7
4. DATABASE DESIGN	8
4.1 Database Collections	8
4.2 Collections Structure	8
4.3 Constraints in MongoDB	9
4.4 Collections Relationships	9
5. SYSTEM DESIGN	10

5.1 System Architecture	10
5.2 Modules Description	10
5. SYSTEM IMPLEMENTATION	11-15
6. CONCLUSION	16
7. FUTURE ENHANCEMENTS	17
9. REFERENCES	18
APPENDIX (Sample Code)	19-20
APPENDIX (MongoDB schema)	21

ABSTRACT

The **Faculty Leave Management System** is a full-stack, web-based application designed to streamline and automate the process of managing faculty leave requests in an educational institution. It replaces traditional manual methods with an efficient digital solution that enables faculty members to apply for leave, track approval status, and manage leave records in real time. The system allows faculty to submit leave applications, view leave balances, and receive notifications regarding approvals or rejections. Administrators and department heads can review, approve, or reject requests, ensuring proper workflow and transparency.

Developed using the **MERN stack (MongoDB, Express.js, React, and Node.js)**, the application ensures efficient data management, fast processing, and a responsive user interface. It incorporates secure authentication using **JWT (JSON Web Token)** to protect user data and maintain system integrity.

Additionally, the system provides features such as leave history tracking, role-based access control, and automated record maintenance, reducing paperwork and administrative effort. With its scalable architecture and user-friendly design, the Faculty Leave Management System improves operational efficiency and provides a reliable, modern solution for managing faculty leave processes.

LIST OF FIGURES

Figure no.	Figure Title	Page no.
1	Registration Page of Faculty Leave Management System	11
2	Employee Login Screen of FLMS	11
3	Admin Dashboard Interface of FLMS	12
4	Faculty Leave Request Form Interface	12
5	Leave Application Submission Page	13
6	Faculty Leave History and Status Tracking Interface	14
7	Admin Panel for Reviewing and Managing Leave Requests	15

LIST OF TABLES

Table no.	Table Title	Page no.
1	Users (Faculty/Admin) MongoDB Collection	8
2	Leave Requests MongoDB Collection	8
3	Departments MongoDB Collection	8
4	Leave Types MongoDB Collection	8
5	Leave Records / History MongoDB Collection	9

ACRONYMS

- **MERN** : MongoDB, Express.js, React, Node.js
- **JWT** : JSON Web Token
- **API** : Application Programming Interface
- **UI** : User Interface
- **UX** : User Experience
- **HTTP** : HyperText Transfer Protocol
- **WS** : WebSocket
- **RTC** : Real-Time Communication
- **DB** : Database
- **ODM** : Object Data Modeling
- **P2P** : Peer-to-Peer
- **REST** : Representational State Transfer

1. INTRODUCTION

1.1 Brief Information about the Project

The Faculty Leave Management System (FLMS) is a full-stack, web-based application developed to simplify and automate the process of managing leave requests within an educational institution. It provides a centralized platform where faculty members can apply for leave, view their leave status, and maintain leave records efficiently. The system enables administrators and department heads to review, approve, or reject leave applications in a structured and transparent manner. It also supports features such as leave history tracking, role-based access, and real-time updates, improving communication between faculty and management. This system replaces traditional manual leave management methods with a more efficient, accurate, and time-saving digital solution. It enhances data organization, reduces paperwork, and ensures better monitoring of faculty availability. The application is developed using the MERN stack, utilizing MongoDB for database management, Express.js and Node.js for backend operations, and React for building an interactive user interface. It also implements secure authentication using JWT (JSON Web Token) to ensure data security and authorized access. Overall, the Faculty Leave Management System is designed to be user-friendly, scalable, and reliable, making it an effective solution for managing faculty leave processes in modern educational institutions.

1.2 Motivation and Contribution of Project

In many educational institutions, faculty leave management is still handled manually or through partially digital systems, which leads to inefficiencies, delays, and lack of transparency. Traditional methods such as paper-based applications or unorganized digital records make it difficult to track leave status, maintain accurate records, and ensure smooth communication between faculty and administration. These limitations motivated the development of the **Faculty Leave Management System (FLMS)**.

- Providing a **centralized platform** for managing faculty leave requests
- Reducing dependency on **manual paperwork and disconnected systems**
- Improving the **efficiency of leave approval workflows**
- Enabling **secure user authentication and role-based access control**
- Ensuring **transparency through real-time status tracking of leave applications**
- Maintaining **accurate and organized leave records** for future reference

This system simplifies the leave management process, reduces administrative workload, and enhances overall efficiency by delivering a reliable and scalable digital solution for managing faculty leave operations.

1.3 Objectives of the Project

The main objectives of the **Faculty Leave Management System (FLMS)** are:

- To develop a **user-friendly web-based application** for managing faculty leave efficiently
- To enable faculty members to **apply for leave and track their leave status** in real time
- To provide functionalities for **leave request submission, approval, and rejection** by administrators
- To maintain **accurate leave records and history** for future reference
- To ensure **secure user authentication and role-based access control** using JWT and database systems
- To reduce **manual paperwork and administrative workload** through automation

1.4 Scope of the Project

The **Faculty Leave Management System (FLMS)** can be used in educational institutions such as colleges and universities to efficiently manage faculty leave processes. It supports operations such as submitting leave requests, approving or rejecting applications, maintaining leave records, and tracking leave status. The system provides a user-friendly web-based interface for faculty members, administrators, and department heads to handle leave-related activities in a centralized manner. It also ensures proper record management, reduces manual errors, and improves communication between staff and management.

The application can be further extended to include features such as attendance integration, notifications, report generation, and analytics for better decision-making. Overall, the system enhances administrative efficiency and provides a reliable digital solution for managing faculty leave processes.

2. SYSTEM ANALYSIS

2.1 Functional Requirements

Functional requirements define the core operations and functionalities that the Faculty Leave Management System (FLMS) must perform to ensure efficient leave management and smooth interaction between faculty and administration. The functional requirements of the system are:

- **User Registration and Login**

The system should allow faculty members and administrators to register and log in using valid credentials. It should ensure secure authentication using JWT and support basic profile management.

- **Leave Application Management**

The system should allow users to create, update, and delete servers and Channels. It should support organizing communication through structured groups.

- **Real-Time Messaging**

The system should allow users to send, edit, and delete messages in real-time. It should support message reactions and file sharing.

- **Leave Approval System**

The system should enable administrators or department heads to review, approve, or reject leave applications with appropriate remarks. .

- **Leave Tracking and History**

The system should allow users to view the status of their leave requests and maintain a history of all past leave records..

- **Leave Types and Balance Management**

The system should manage different leave categories and allow tracking of available leave balance for each faculty member

- **Notifications and Updates**

The system should provide real-time notifications regarding leave application status, approvals, or rejections.

- **Role-Based Access Control**

The system should restrict access based on user roles (faculty, admin), ensuring proper authorization and data security.

2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes and performance standards that the **Faculty Leave Management System (FLMS)** must satisfy to ensure reliability, efficiency, and usability..

- **Performance**

The system should respond quickly to operations like sending and receiving messages, loading chats, and switching between channels without noticeable delay.

- **Security**

Only authorized users should access and modify data using proper authentication and access control.

- **Usability**

The system should be simple and easy to use, allowing users to perform tasks without difficulty.

- **Reliability**

The system should operate consistently and handle errors without data loss.

- **Scalability**

The system should handle increasing data and users without affecting performance.

2.3 Requirements Specification

This section specifies the hardware and software needed to run the system.

2.3.1 Minimum Hardware Requirements

- Processor: Intel i3 or above
- RAM: 4 GB (minimum)
- Hard Disk: 20 GB free space
- Monitor: Standard display
- Keyboard and Mouse

2.3.2 Software Requirements

- **Operating System:** Windows 7/10/11
- **Frontend:** React.js

- **Backend:** Node.js with Express.js
- **Programming Language:** JavaScript
- **IDE:** VS Code
- **Database:** MongoDB
- **Connectivity:** Mongoose / REST API

3. TECHNOLOGY DESCRIPTION

3.1 Programming Language

3.1.1 Introduction to JavaScript

JavaScript is a high-level, versatile programming language widely used for developing web-based applications. It supports both client-side and server-side development, making it a core technology for full-stack applications. It is known for its flexibility, asynchronous nature, and ability to create dynamic and interactive user interfaces.

In the Cyber-Chat application, JavaScript is used to implement the core functionality of the system. It handles operations such as user interaction, real-time messaging, server communication, and data processing. JavaScript provides features like event-driven programming, asynchronous handling, and efficient memory usage, which help in building responsive and scalable applications..

One of the main advantages of JavaScript is that it can run in both the browser and the server using Node.js. This makes it suitable for developing full-stack applications like Cyber-Chat.

3.2 Framework / Libraries

3.2.1 Introduction to React (Frontend Framework)

React is a popular JavaScript library used for building user interfaces, especially for single-page applications. It allows developers to create reusable components and efficiently update the user interface using a virtual DOM.

In this project, React is used to build the frontend of the Cyber-Chat application. It allows users to interact with the system through features such as:

- 1.4.1.1.1 User Authentication (Login / Signup)
- 1.4.1.1.2 Chat Dashboard Interface
- 1.4.1.1.3 Server and Channel Navigation
- 1.4.1.1.4 Messaging and Notifications
- 1.4.1.1.5 Voice and Video Controls

React provides a fast, interactive, and user-friendly interface, making the application responsive and efficient.

3.2.2 Introduction to Node.js

Node.js is a runtime environment that allows JavaScript to run on the server side, while Express.js is a lightweight web framework used to build APIs and handle server-side logic.

In this project, Node.js and Express.js are used to develop the backend of the Cyber-Chat application. They handle operations such as:

- Managing API requests and responses
- Handling user authentication and authorization
- Processing real-time communication using Socket.IO
- Managing data flow between frontend and database

These technologies make the backend efficient, scalable, and capable of handling multiple users simultaneously.

3.3 Database Technology

The Distributed-Messaging System uses a NoSQL database for storing and managing user, message, and server data in a flexible and scalable manner. The database ensures efficient data storage, retrieval, and real-time updates, which are essential for communication-based systems.

In this project, MongoDB is used as the backend database, and Mongoose is used for connecting the application to the database and managing data models.

.

3.3.1 Mongoose (Object Data Modeling)

Mongoose is an Object Data Modeling (ODM) library used to interact with MongoDB in Node.js applications. It provides a schema-based solution to define data structures and perform database operations.

In the Employee Management System, JDBC is used to connect the Java application with the Oracle database. It performs operations such as:

- Define schemas for users, messages, servers, and other entities
- Perform database operations such as insert, update, delete, and retrieve data
- Ensure data validation and consistency
- Manage relationships between different collections

. Mongoose acts as a bridge between the application and MongoDB, ensuring smooth data handling.

3.3.2 MongoDB

MongoDB is a NoSQL database used for storing data in the form of documents. It is designed for scalability, flexibility, and high performance, making it suitable for modern web applications.

In the Distributed-Messaging System, MongoDB is used to store user details, messages, server data, and other application-related information. It supports dynamic schemas and allows efficient handling of large volumes of real-time data.

3.4 Tools & IDEs

3.4.1 Visual Studio Code

Visual Studio Code (VS Code) is used as the development environment for the Employee Management System. It is a lightweight code editor that supports Java programming through extensions. VS Code provides features such as syntax highlighting, code completion, and debugging support, which help improve coding efficiency. Its simple and user-friendly interface makes it suitable for developing and testing the application.

4. DATABASE DESIGN

4.1 Database Collections

The **Faculty Leave Management System (FLMS)** uses **MongoDB**, a NoSQL database, to store data in a flexible and scalable manner. The database consists of multiple collections such as Users, Leave Requests, Departments, Leave Types, and Leave Records.

- **The Users collection** stores details of registered users, including faculty members and administrators in the system.
- **The Leave Requests collection** stores information about leave applications submitted by faculty, including dates, type of leave, and status.
- **The Departments collection** stores details of different departments within the institution to organize faculty members.
- **The Leave Types collection** stores various categories of leave such as casual leave, sick leave, and earned leave.
- **The Leave Records collection** maintains the history of all approved, rejected, and pending leave applications for tracking and reference.

4.2 Table Structure

4.2.1 Users Collection

```
const userSchema = new mongoose.Schema({  
  name: String,  
  email: String,  
  password: String,  
  role: String, // faculty or admin  
  department: String  
});
```

4.2.2 Leave Request Collection

```
const leaveRequestSchema = new mongoose.Schema({  
  user_id: String,  
  leave_type: String,  
  start_date: Date,  
  end_date: Date,  
  reason: String,  
  status: String, // pending, approved, rejected  
  appliedAt: Date  
});
```

4.2.3. Department Collection

```
const departmentSchema = new mongoose.Schema({  
  department_name: String,  
  hod_id: String,  
  createdAt: Date  
});
```

4.2.4 Leave Types Collection

```
Const leaveTypeSchema = new mongoose.Schema({  
  leave_name: String, // casual, sick, earned  
  max_days: Number,  
  description: String
```

```
});
4.2.5 Leave Records Collection
const leaveRecordSchema = new mongoose.Schema({
  user_id: String,
  leave_request_id: String,
  status: String,
  approved_by: String,
  updatedAt: Date
});
```

4.3 Constraints in MongoDB

MongoDB provides basic constraints to maintain data integrity, but they are not as strict as relational databases.

- Each document contains a unique `_id` field which acts as a primary key.
- Fields like email can be made unique to avoid duplicate user accounts.
- Important fields such as username, password, and message are required and cannot be left empty.
- Data types are enforced using Mongoose schemas to ensure valid data storage.
- Relationships between collections are maintained using references such as `user_id`, `server_id`, and `channel_id`.
- These constraints help in maintaining consistency and preventing invalid or duplicate data in the system.

4.4 Collections Relationships

The **Faculty Leave Management System (FLMS)** maintains relationships between collections using reference fields to ensure proper data organization and connectivity.

- **Each department can have multiple users (faculty members) (*One-to-Many*)**
One department contains many faculty members.
- **Each user can submit multiple leave requests (*One-to-Many*)**
A faculty member can apply for multiple leaves over time.
- **Each leave request belongs to one user (*Many-to-One*)**
Every leave request is linked to a specific faculty member.
- **Each leave request is associated with one leave type (*Many-to-One*)**
Each request corresponds to a specific leave category (casual, sick, etc.).
- **Each leave request has one leave record (*One-to-One*)**
Tracks the final status (approved/rejected) of the request.
- **Each admin/HOD can approve multiple leave requests (*One-to-Many*)**
One admin can handle multiple leave approvals.

5. SYSTEM DESIGN

5.1 System Architecture

The Distributed-Messaging System follows a three-layer architecture:

1. Presentation Layer (User Interface)

- o Developed using React.js
- o Allows users to interact with the system through a web interface

2. Application Layer (Business Logic)

- o Implemented using Node.js and Express.js
- o Handles user requests and processes data
- o Performs operations like sending messages, creating channels, and managing servers

3. Data Layer (Database)

- o Stores user, message, server, and payment data
- o Managed using MongoDB

This architecture ensures better organization, scalability, and maintainability.

5.2 Modules Description

User Module

- Handles user registration and login for faculty and administrators
- Manages secure authentication using JWT
- Stores and retrieves user profile details such as name, department, and role

Leave Request Module

- Allows faculty members to apply for leave
- Enables editing or canceling leave applications
- Stores leave request details such as dates, type, and reason

Leave Approval Module

- Allows admin/HOD to review leave requests
- Approves or rejects applications with remarks
- Updates the status of leave requests

Leave Management Module

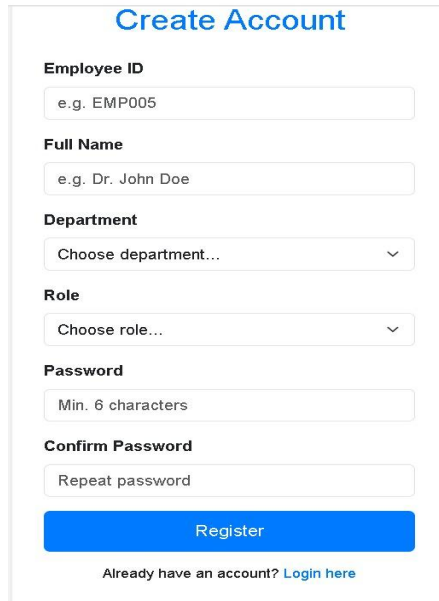
- Maintains leave records and history
- Tracks leave balance for each faculty member
- Provides reports on leave usage

Notification Module

- Sends notifications for leave application status (approved/rejected)
- Alerts users about new requests or updates
- Improves communication between faculty and administration

6. SYSTEM IMPLEMENTATION

Registration Page Interface:



The registration form is titled "Create Account" in blue. It contains several input fields: "Employee ID" with a placeholder "e.g. EMP005", "Full Name" with a placeholder "e.g. Dr. John Doe", "Department" and "Role" as dropdown menus with "Choose department..." and "Choose role..." respectively, "Password" with a placeholder "Min. 6 characters", and "Confirm Password" with a placeholder "Repeat password". A blue "Register" button is at the bottom, followed by a link "Already have an account? Login here".

Create Account

Employee ID
e.g. EMP005

Full Name
e.g. Dr. John Doe

Department
Choose department... ▾

Role
Choose role... ▾

Password
Min. 6 characters

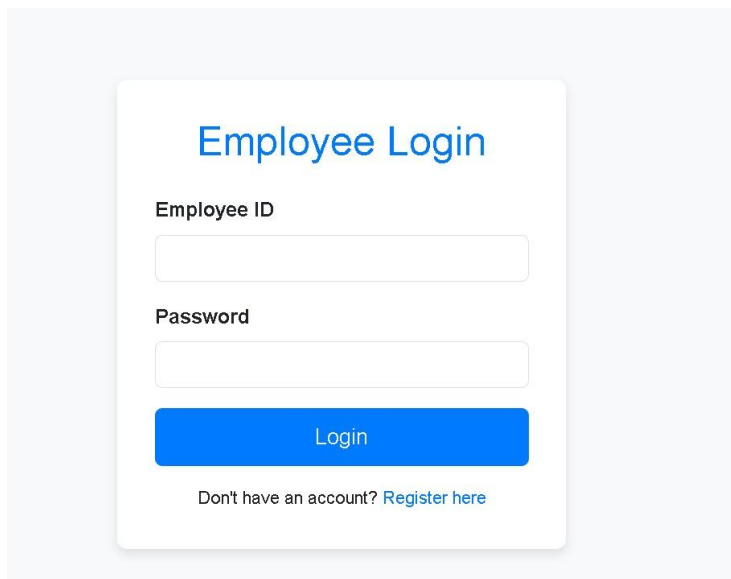
Confirm Password
Repeat password

Register

Already have an account? [Login here](#)

Figure 1: Registration (Create Account) Page Interface

User Login Screen:



The login form is titled "Employee Login" in blue. It contains two input fields: "Employee ID" and "Password". A blue "Login" button is at the bottom, followed by a link "Don't have an account? Register here".

Employee Login

Employee ID

Password

Login

Don't have an account? [Register here](#)

Figure 2: Employee Login Interface

Admin Dashboard Interface:

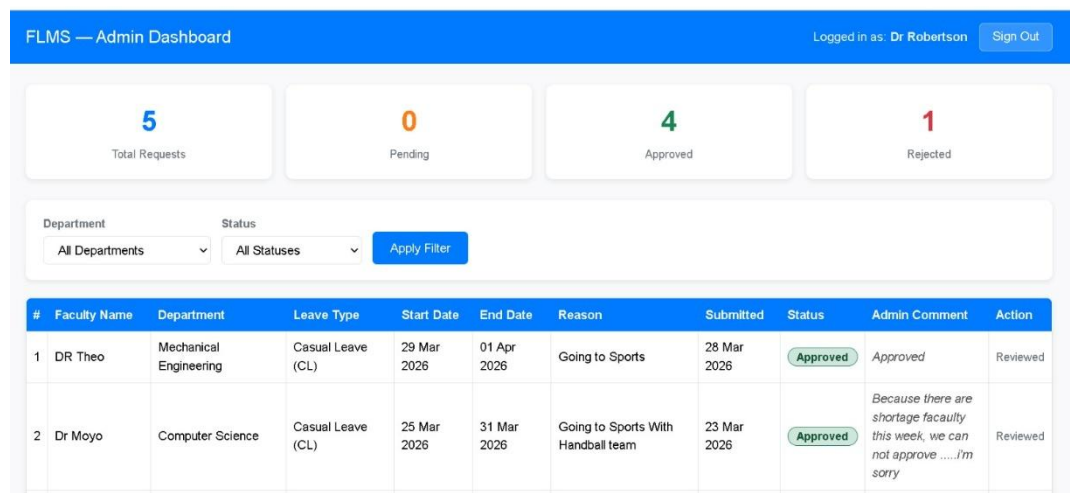


Figure 3: Admin Dashboard Showing Leave Requests Overview

Leave Request Form Interface:

The screenshot shows the "Faculty Leave Request Form" in the FLMS Portal. The header bar includes the "FLMS Portal" logo and the user "Dr. Alice Johnson" with a "Sign Out" button. The form title is "Faculty Leave Request Form" with a subtext "Complete all required fields and submit your request for approval." The form is divided into two sections: "PERSONAL INFORMATION" and "LEAVE DETAILS". The "PERSONAL INFORMATION" section has two fields: "FULL NAME *" (filled with "Dr. Alice Johnson") and "DEPARTMENT *" (a dropdown menu showing "Computer Science"). The "LEAVE DETAILS" section has one field: "TYPE OF LEAVE *" (a dropdown menu showing "Choose leave type...").

Figure 4: Faculty Leave Request Form

Leave Application Submission Section:

START DATE *

dd-mm-yyyy

END DATE *

dd-mm-yyyy

REASON FOR LEAVE *

Briefly describe the reason for your leave...

COVERAGE PLAN (Optional)

BACKUP FACULTY / COVERAGE ARRANGEMENTS

e.g., Prof. Smith will cover my CS101 class on Mon/Wed.

→ Submit Leave Request

Please fill out all required fields marked with *.

Figure 5: Leave Details and Submission Interface

Leave History Interface

My Leave History							
Track the status of all your submitted leave requests.							
#	LEAVE TYPE	START DATE	END DATE	REASON	SUBMITTED	STATUS	ADMIN COMMENT
1	Casual Leave (CL)	31 Mar 2026	02 Apr 2026	International Conference	29 Mar 2026	Pending	—
2	Casual Leave (CL)	25 Mar 2026	31 Mar 2026	Going to Sports With Handball team	23 Mar 2026	Approved	Because there are shortage faculty this week, we can not approveI'm sorry
3	Sick Leave (SL)	22 Mar 2026	30 Mar 2026	I have an operation scheduled on this day	21 Mar 2026	Approved	—
4	Casual Leave (CL)	17 Mar 2026	20 Mar 2026	Vaccation	17 Mar 2026	Rejected	—

Admin Review Interface:

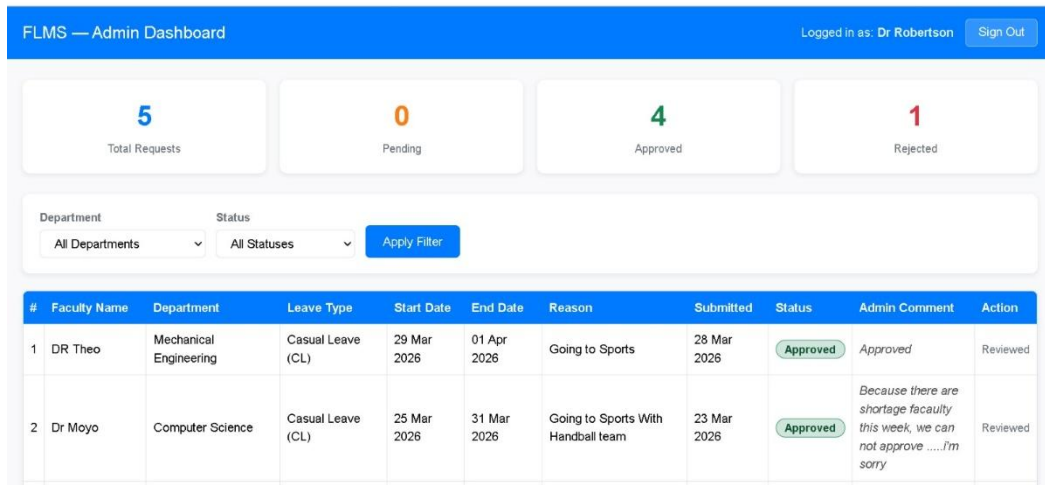


Figure 7: Admin Panel for Reviewing Leave Requests

7. CONCLUSION

The **Faculty Leave Management System (FLMS)** has been successfully developed to provide a modern and efficient solution for managing faculty leave processes in educational institutions. The system replaces traditional manual methods with an integrated digital platform, thereby improving efficiency, reducing delays in approval, and enhancing transparency.

The application provides essential functionalities such as user registration and login, leave application submission, approval and rejection of requests, and leave history tracking through a user-friendly web interface. By using technologies like the **MERN stack** and **JWT authentication**, the system ensures efficient performance, secure data handling, and reliable access control.

Overall, the system improves the efficiency of leave management by automating the entire process and maintaining accurate records in a centralized database. It also provides a scalable solution that can be further enhanced in the future by integrating features such as attendance management, notification systems, mobile application support, and advanced reporting tools.

In conclusion, the Faculty Leave Management System meets the project objectives and serves as an effective platform for managing faculty leave requests, improving administrative workflow, and ensuring smooth institutional operations.

8. FUTURE ENHANCEMENTS

The **Faculty Leave Management System (FLMS)** developed in this project provides essential functionalities for managing faculty leave efficiently. However, the system can be further enhanced by adding advanced features to improve its performance, usability, and scalability.

Mobile Application System :

Developing a mobile application to allow faculty and administrators to access the system anytime and anywhere.

Advanced Security Features:

Implementing features like two-factor authentication (2FA), data encryption, and secure role-based access for enhanced data protection.

AI-Based Features:

Integrating AI to provide leave approval suggestions, detect unusual leave patterns, and automate decision-making support.

Attendance Integration:

Linking the system with attendance management to automatically update leave records and track faculty availability.

Automated Notifications:

Sending email/SMS notifications for leave application status, reminders, and approvals.

Report Generation and Analytics:

Generating detailed reports on leave usage, trends, and department-wise statistics for better decision-making.

Improved User Interface:

Enhancing the system design to provide a more interactive and user-friendly experience.

9. REFERENCES

The following references were used during the development of the Faculty Leave Management System.

1. MongoDB Documentation: <https://www.mongodb.com/docs/>
2. React Documentation: <https://react.dev/>
3. Node.js Documentation, <https://nodejs.org/en/docs/>
4. Express.js Documentation: <https://expressjs.com/>
5. Socket.IO Documentation, <https://socket.io/docs/>
6. WebRTC Documentation (MDN), https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
7. Stripe Documentation, <https://stripe.com/docs/>
8. Razorpay Documentation, <https://razorpay.com/docs/>
9. JavaScript Tutorial – W3Schools, <https://www.w3schools.com/js/>
10. MERN Stack Guide – GeeksforGeeks, <https://www.geeksforgeeks.org/mern-stack/>

APPENDIX

A.1 Sample Code

Authentication code:

The following code implements user authentication using JSON Web Tokens (JWT) in a Node.js application. It verifies user credentials and generates a token for secure access to protected resources.

Auth.js:

```
import jwt from "jsonwebtoken";

const auth = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (authHeader) {
    const token = authHeader.split(" ")[1];
    jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
      if (err) {
        return res.status(403).json("Token is not valid!");
      }
      req.user = { id: decoded.id, _id: decoded.id }; // Set both to be safe
      next();
    });
  } else {
    return res.status(401).json("You are not authenticated!");
  }
};

export default auth;
```

CheckMember.js: The following code verifies whether a user is a valid member by checking the JSON Web Token (JWT) provided in the request. It ensures that only authenticated members can access protected resources.

```
. import Server from '../models/Server.js';
import Channel from '../models/Channel.js';

const checkMember = async (req, res, next) => {
  try {
```

```

let serverIdStr = req.params.serverId || req.params.id;
// If the route only has a channelId (e.g. GET /messages/:channelId)

if (!serverIdStr && req.params.channelId) {
  const channel = await Channel.findById(req.params.channelId);
  if (!channel) return res.status(404).json({ message: "Channel not
found" });
  serverIdStr = channel.serverId.toString();
}

// If serverId is in the body (e.g. POST /messages)
if (!serverIdStr && req.body.serverId) {
  serverIdStr = req.body.serverId;
}

if (!serverIdStr) {
  return res.status(400).json({ message: "Server ID is required to
authorize access" });
}

const server = await Server.findById(serverIdStr);
if (!server) {
  return res.status(404).json({ message: "Server not found" });
}

const isMember = server.members.some(member =>
member.user.toString() === req.user.id);

if (!isMember) {
  return res.status(403).json({ message: "Forbidden: You are not a
member of this server." });
}

// Pass server down to avoid redundant lookups in subsequent
controllers
req.server = server;
next();
} catch (err) {
  console.error("Auth Middleware [checkMember] Error:", err);
  res.status(500).json({ message: "Internal server error authorizing
member." });
}
};

export default checkMember;

```

A.2 MongoDB Schema

Create Channel schema(Channel.js) :

```
import mongoose from "mongoose";

const leaveRequestSchema = new mongoose.Schema({
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true
  },
  leaveType: {
    type: String,
    required: true
  },
  startDate: {
    type: Date,
    required: true
  },
  endDate: {
    type: Date,
    required: true
  },
  reason: {
    type: String,
    required: true
  },
  status: {
    type: String,
    enum: ["pending", "approved", "rejected"],
    default: "pending"
  },
  approvedBy: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User"
  }
}, { timestamps: true });

export default mongoose.model("LeaveRequest", leaveRequestSchema);
```